

High Performance and Distributed Computing for Big Data

Course presentation

*Miquel A. Senar, miquelangel.senar@uab.cat, miquelangel.senar@urv.cat,
Sandra Mendez, sandra.mendez@uab.cat
Francesc Solsona, francesc.solsona@udl.cat*

10/02/2026

Course Overview

1. Algorithmic Techniques and Programming (Sandra Méndez)
 - Algorithms
 - Advance
2. High Performance Computing Infrastructures (Miquel A. Senar / S. Mendez)
 - HPC and Performance
 - System middleware: Batch Queue Systems
 - Virtualization and Containers
 - Workflow Management Systems
3. Cloud Systems (Francesc Solsona)
 - Cloud Computing Platforms
 - Big Data Analysis

Course Schedule (tentative)

WEEK	UNIT/TOPIC	GRADED ACTIVITIES	WEIGHT	DEADLINE ACTIVITIES
From 10/02 to 31/03	Unit 2: System Middleware (HPC and Batch Queue Systems). Virtualization and Containers.	Activity 1.	15%	TBA
From 17/02 to 26/05	Unit 1. Algorithms & Programming	Activity 2 & 3	35%	TBA
From 7/04 to 26/05	Unit 2. Workflow Management Systems	Activity 4	15%	TBA
From 18/04 to 29/04	Unit 3. Cloud Systems	Activity 5	15%	TBA
From 6/05 to 27/05	Unit 3. Big Data Management	Activity 6	15%	TBA

Assessment

- Activities of each Unit/Topic consisting of practical exercises that must be completed individually or by groups of 2 people maximum
- Submissions will include written reports plus programs/scripts,...
- Late submissions will be penalized
- Individualized interviews on the exercises delivered (weeks from June 11 to 17). Activity grade will be mainly based on these interviews plus submitted documents. You can explain/adapt your programs to our requests
- All activities should have a minimum score of 3 to pass the subject
- No final exam

Course Tenets (Units 1 and 2)

- Keep it simple, work gradually (increasing complexity/functionality) and keep it modular.
- Some masterclass sessions + Flipped-classroom and hands-on exercises
 - learning at your own pace
 - personalized online resources
 - learning by problem solving in the “context” of biomedical data-analysis & bioinformatics
- Focus on programming, methodologies, and techniques;
 - languages, tools or frameworks are contingent but relevant (stick to rules).
- Solving problems Work at your own pace (alone or with your classmates)
 - ChatGPT, StackOverflow might help but not replace
 - Questions via email in case of getting stuck ;-)
 - miquelangel.senar@uab.cat

A Word of Advise

DON'T PANICK !

- This is **crash course** on many computer skills.
- It's easy to feel **overwhelmed** by the amount of information to process, concepts to learn, and tasks to deliver.
 1. Stop, relax, and come up with a **plan** (decompose the problem, isolate errors)
 2. Identify inputs and outputs (**What do I want it to do?**)
 3. Design small tests and progress **incrementally**
 4. **Code & test** until you reach a solution
 5. **Ask questions!**, Get answers!

Introductory Questionnaire (at moodle)

1. Previous studies: bachelor degrees, masters, others...
2. Master's dedication:
 - a. full time ; part time (studying + working; please specify time availability)
 - b. Regular attendance of synchronous classes
 - c. Partial attendance at synchronous classes (scattered days or incomplete session)
 - d. Will follow the course mainly through the recordings
3. Computing background:
 - a. My personal computer (Windows, Linux, Mac; specify if you use virtualized systems)
 - b. Computing skills (basic, medium, advanced)
 - i. Linux commands (file management, folder management, program execution,...)
 - ii. Linux tools (sed, awk, grep,...)
 - iii. Linux Bash programming
 - iv. Python programming
 - v. Python ecosystem (specify which modules you know: numpy, pandas, pytorch,...)
 - vi. Other languages (R, C/C++,...)
 - vii. Algorithmic techniques (Recursion, Dynamic Programming, Divide & Conquer,...)
4. Other comments (personal interests, expectations....)

Motivation

- Biomedical experiments characteristics
 - Large amount of (eventually, heterogeneous) data sets
 - Combination of external tools and user-developed programs
 - Workflow paradigm
- Biomedical experiment design:
 - Based on computing tools (automatic processing vs manual operation)
 - Repeatability (same team, same experiment setup, same result)
 - Replicability (different team, same experiment setup, same result)
 - Reproducibility (different team, different experiment setup, new results)
 - Incremental analysis
 - High Performance Computing (HPC) and High Throughput Computing (HTC)

Motivation: Why HPC/Cloud Systems?

Technology improvement: Moore's Law

Year	Transistors	CPU model
1975	3 000	6502
1979	30 000	8088
1985	300 000	386
1989	1 000 000	486
1995	6 000 000	Pentium Pro
2000	40 000 000	Pentium 4
2005	100 000 000	2-core Pentium D
2008	700 000 000	8-core Nehalem
2014	6 000 000 000	18-core Haswell
2017	20 000 000 000	32-core AMD Epyc
2019	40 000 000 000	64-core AMD Rome

Why HPC Systems?

Performance improvement: Clock freq.

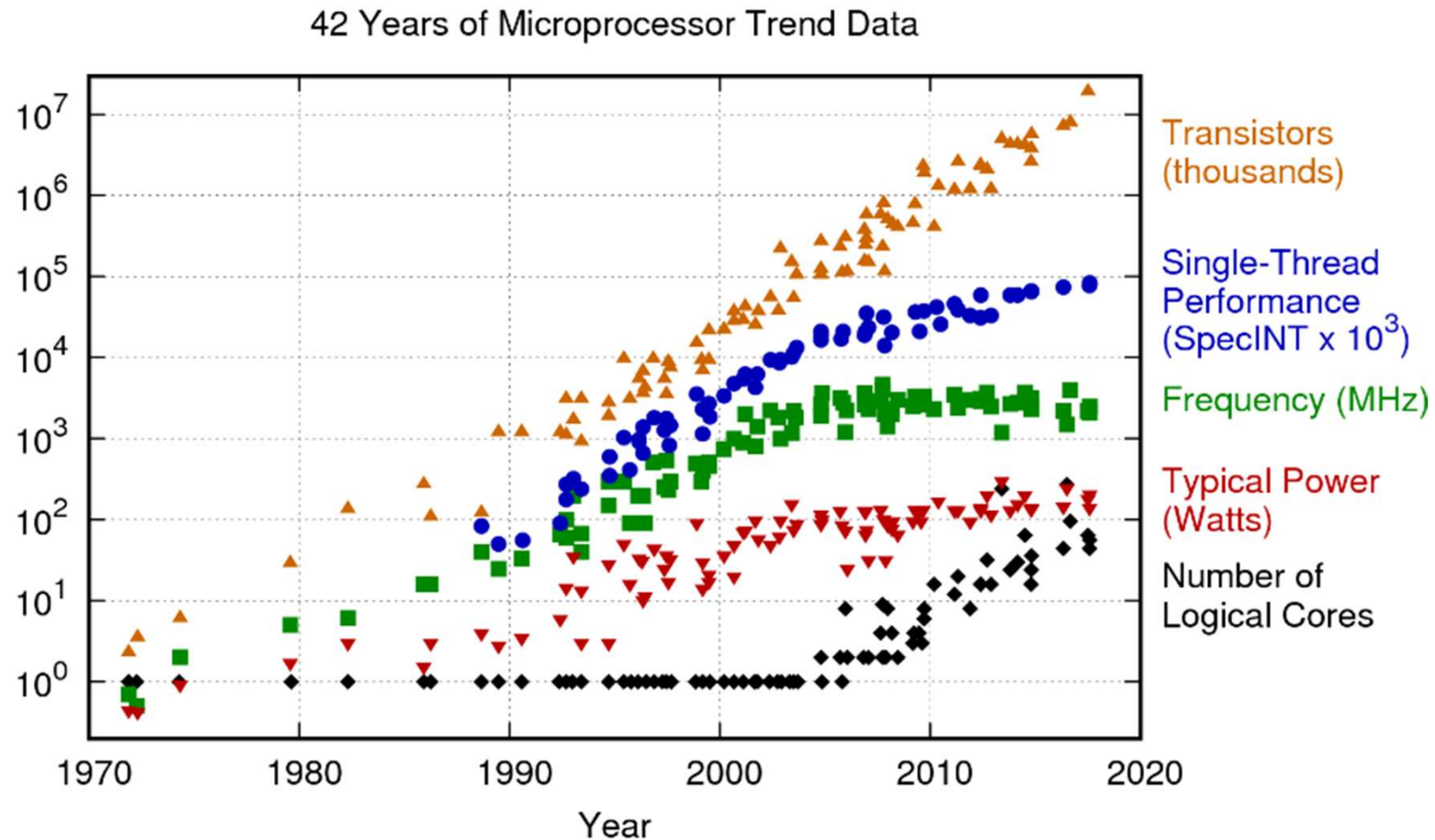
Year	Transistors	Clock speed	CPU model
1975	3 000	1 000 000	6502
1979	30 000	5 000 000	8088
1985	300 000	20 000 000	386
1989	1 000 000	20 000 000	486
1995	6 000 000	200 000 000	Pentium Pro
2000	40 000 000	2 000 000 000	Pentium 4
2005	100 000 000	3 000 000 000	2-core Pentium D
2008	700 000 000	3 000 000 000	8-core Nehalem
2014	6 000 000 000	2 000 000 000	18-core Haswell
2017	20 000 000 000	3 000 000 000	32-core AMD Epyc
2019	40 000 000 000	3 000 000 000	64-core AMD Rome

Why HPC Systems?

Performance improvement: instruction latency (FMUL operation)

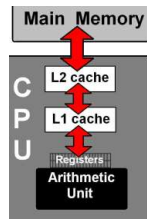
Year	Clock cycles	CPU model
1980	100	8087
1987	50	387
1993	3	Pentium
2018	5	Coffee Lake

Why HPC Systems?

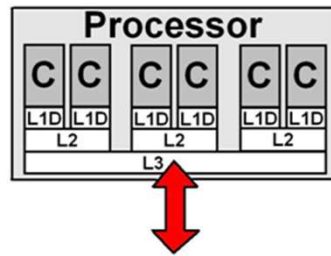


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

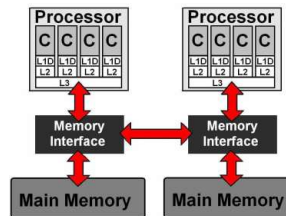
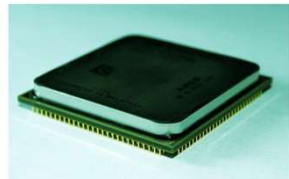
Modern Computer Architectures



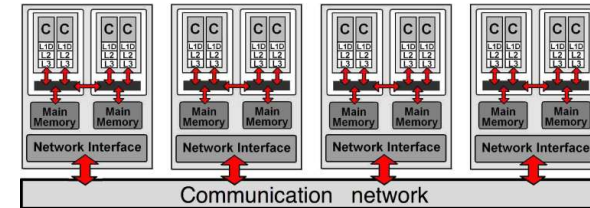
CPU



CPU multicore



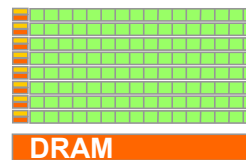
Node (rack)
multiprocessor



Cluster / Supercomputer



Accelerators /
Coprorocessors



GPU

- CPU Coprocessor
- many threads in parallel
- private memory

Cluster Systems: Basic Architecture

